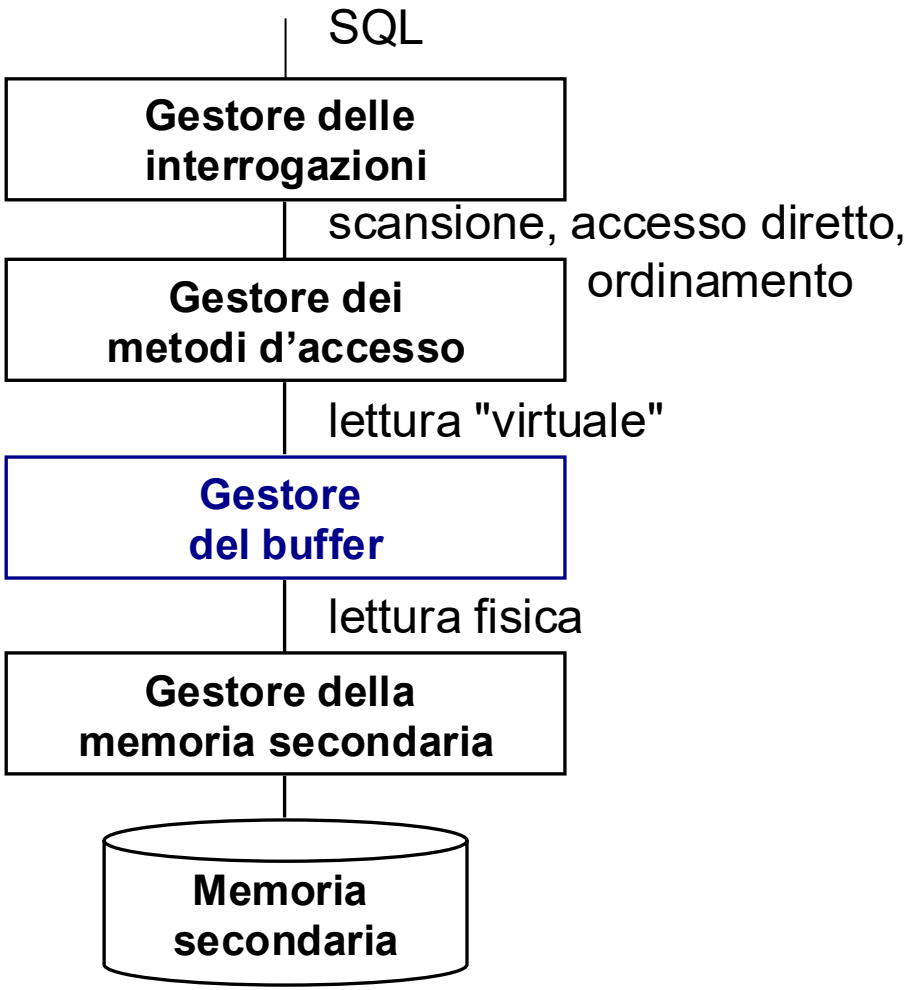
Gestore degli accessi e delle interrogazioni



In SimpleDB

Remote: riceve le richieste dal client e passa SQL a Planner

Planner: chiama Parser e determina piano e lo passa al Query

Parser: analisi sintattica

Query: riceve il piano e chiama il Record per ogni tabella

Metadata: gestisce gli schemi delle tabelle

Record: gestisce i blocchi per i record delle tabelle

Transaction: gestisce la concorrenza

Buffer: mantiene in memoria le pagine per limitare gli accessi

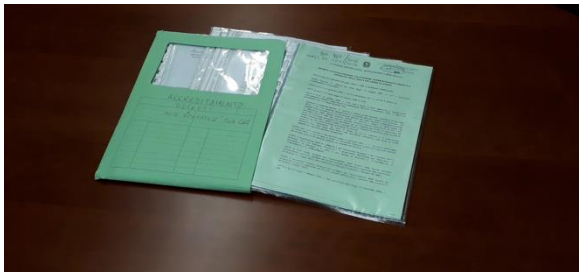
Log: tiene traccia delle operazioni, per l'affidabilità

File: legge e scrive le pagine su disco

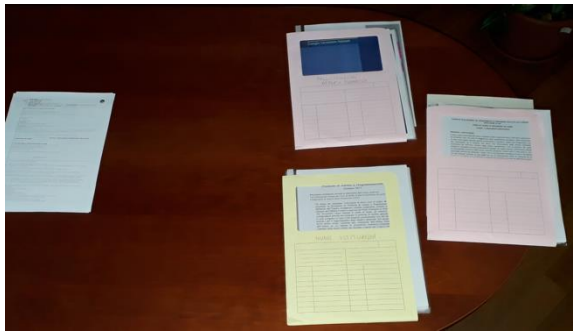
Il buffer, una metafora

- I miei documenti sono nel classificatore

- Per lavorarci, li sposto sulla scrivania

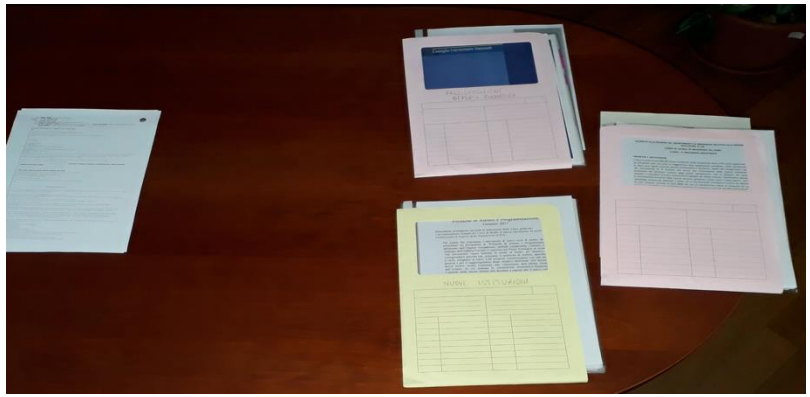


- Se debbo lavorare a più argomenti, tengo più cartelline sulla scrivania



Ancora la metafora

- …
- Se debbo lavorare a più argomenti, tengo più cartelline sulla scrivania



- Ad un certo punto la scrivania si riempie e non riesco più a lavorare
- Debbo rimettere a posto

Euristiche

- Come scelgo quale rimettere a posto?
  - Quella che penso che non mi servirà prossimamente
  - Assumo che sia quella che non uso da più tempo
- Con uno spirito diverso, ma con l'idea di algoritmo semplice, la metafora dell'armadio

Un’altra metafora

- Prendo ciò che mi serve
- Metto a posto inserendo all’estremità destra
- Domande:
  - Dove stanno le cose che ho usato più di recente?
  - Dove stanno a fine inverno i giubbetti estivi?
  - Dove stanno le cose che non mi servono (e che magari dismetto se mi serve spazio)?

Gestione dei buffer (“buffer management”)

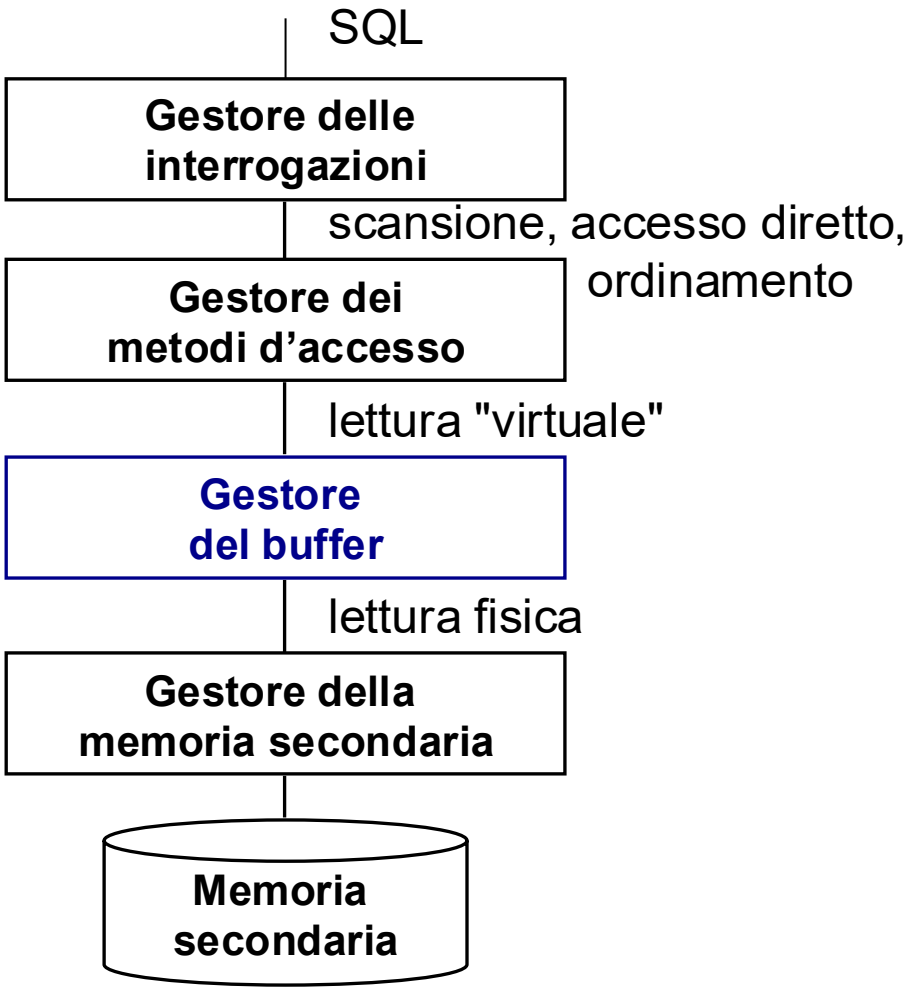
- **Buffer**:
  - area di memoria centrale, gestita dal DBMS (preallocata) e condivisa fra le transazioni
  - organizzato in **pagine** di dimensioni pari o multiple di quelle dei blocchi di memoria secondaria (1KB-100KB)
  - è importantissimo per via della grande differenza di tempo di accesso fra memoria centrale e memoria secondaria

- NB, esiste spesso anche un buffer del disco (chiamato anche cache del disco) che è chiaramente una cosa diversa

Scopo della gestione del buffer

- Ridurre il numero di accessi alla memoria secondaria
  - leggere e scrivere solo se serve davvero …
  - "tenendo sulla scrivania le cartelline che uso più spesso"

Gestore degli accessi e delle interrogazioni



Scopo della gestione del buffer

- Ridurre il numero di accessi alla memoria secondaria
  - In caso di lettura,
    - se la pagina è già nel buffer, non è necessario leggere da memoria secondaria
  - In caso di scrittura,
    - il gestore del buffer può decidere di differire la scrittura fisica (ammesso che ciò sia compatibile con la gestione dell'affidabilità – vedremo più avanti)
- Sfrutta la "località dei dati":
  - è alta la probabilità di riutilizzare dati utilizzati di recente (vedi la metafora dell'armadio)
- La gestione dei buffer e la differenza di costi fra memoria principale e secondaria possono suggerire algoritmi specifici.

Dati gestiti dal buffer manager

- Il buffer
- Un direttorio (una tavola) che per ogni pagina mantiene un insieme di informazioni utili, ad esempio
  - un identificatore fisico del blocco (ad esempio il nome del file e il numero del blocco; oppure l'indirizzo fisico del blocco)
  - due variabili di stato:
    - un contatore che indica quanti programmi utilizzano la pagina
    - un booleano che indica se la pagina è “sporca”, cioè se è stata modificata

Funzioni del buffer manager

- Intuitivamente:
  - riceve richieste di lettura e scrittura (di pagine/blocchi)
  - le esegue accedendo alla memoria secondaria solo quando indispensabile e utilizzando invece il buffer quando possibile
  - esegue le primitive
    - ***fix (pin), unfix (unpin), setDirty, force (flush)***
  - in simpleDB (modificato nella versione utilizzata dal 2021)
    - ***pin, unpin, setModified, flush***

Interfaccia offerta dal buffer manager (una possibilità, non l'unica)

- ***fix*** o ***pin****:* richiesta di un blocco da parte di un programma;
- ***setDirty****:* comunica al buffer manager che la pagina è stata modificata e non ancora salvata in memoria secondaria (a seconda delle implementazioni può non essere necessaria; o essere gestita direttamente dalle operazioni di modifica)
- ***unfix*** o ***unpin****:* indica che il programma ha concluso l'utilizzo della pagina
- ***force*** o ***flush***: trasferisce in modo sincrono una pagina in memoria secondaria (serve al gestore dell'affidabilità, chiariremo fra qualche lezione)

"Implementazione" dell'interfaccia offerta dal buffer manager

- ***fix*** o ***pin****:* richiesta di un blocco da parte di un programma;
  - restituisce il numero della pagina in cui è presente il blocco
  - esegue una lettura fisica se il blocco non è nel buffer
  - incrementa il contatore associato al blocco (pagina) stesso
- ***setDirty****:* comunica al buffer manager che la pagina è stata modificata e non ancora salvata in memoria secondaria (a seconda delle implementazioni può non essere necessaria; o essere gestita direttamente dalle operazioni di modifica)
  - tiene traccia dell'informazione
- ***unfix*** o ***unpin****:* indica che il programma ha concluso l'utilizzo della pagina
  - decrementa il contatore associato alla pagina
- ***force*** o ***flush***: trasferisce in modo sincrono una pagina in memoria secondaria
  - scrive su disco

Esecuzione della fix

(vediamola sulla mia scrivania …)

- Cerca la pagina nel buffer (in tutti i casi di successo incrementa il contatore della pagina restituita)
  - se c'è, restituisce l'indirizzo (il numero della pagina nel buffer)
  - altrimenti, cerca nel buffer una pagina "libera" (cioè non associata ad un blocco oppure non utilizzata da nessun altro programma, cioè con contatore zero);
    - se ne trova, ne sceglie una (vedi più avanti i possibili criteri)
      - se è sporca la salva
      - legge il blocco di interesse dalla memoria secondaria e restituisce l'indirizzo della pagina
    - se non la trova, due alternative
      - "**no-steal**": pone l'operazione in attesa
      - "**steal**": seleziona una "vittima", pagina occupata del buffer; scrive i dati della vittima in memoria secondaria (se "dirty"); legge il blocco di interesse dalla memoria secondaria e restituisce l'indirizzo --- ignoriamo e trascuriamo I dettagli

Commenti

- Il buffer manager esegue scritture in due contesti diversi:
  - in modo **sincrono** quando è richiesto esplicitamente con una force/flush (vedremo in seguito, cfr. gestione dell'affidabilità)
  - in modo **asincrono** rispetto alle richieste quando lo ritiene opportuno (o necessario)
    - per liberare una pagina;
    - ma può decidere di anticipare o posticipare scritture per coordinarle e/o sfruttare la disponibilità dei dispositivi

Strategia di rimpiazzo (Buffer replacement strategy)

- Quale che sia la politica (steal o no-steal), si pone l'esigenza di scegliere la pagina libera da associare al blocco (l'ideale sarebbe mantenere nel buffer le pagine che, pur libere, potrebbero essere riutilizzate)
- Strategie (supponiamo politica no-steal, che è l'unica che ci interessa):
  - naif (utilizza per il rimpiazzo la prima pagina libera)
  - FIFO (utilizza per il rimpiazzo la pagina libera che è stata caricata da più tempo)
  - LRU (utilizza per il rimpiazzo la pagina libera che è stata usata meno di recente)
  - clock (fa una scansione, come nel caso naif, ma non dall'inizio, bensì dalla pagina successiva a quella del rimpiazzo precedente)

Esercizio

- Prova parziale del 28/03/2012, domanda 4

10	unpin(70)
11	pin(60)
12	setXXX(60,...)
13	unpin(60)
14	flushAll
15	setXXX(47,...)
16	unpin(47)
17	pin(70)
18	setXXX(70,...)
19	unpin(70)
20	pin(60)
21	unpin(60)
22	pin(70)

Nella figura seguente è schematizzato un piccolissimo buffer con quattro pagine (numerate da 0 a 3), il cui stato viene descritto, per ciascuna pagina, da (i) un intero che indica il numero di pin su di essa (quindi 0 indica che la pagina è libera) (ii) un riferimento al blocco che per ultimo è stato caricato nella pagina; (iii) l'istante in cui è stato effettuato l'ultimo caricamento; (iv) l'istante in cui la pagina è stata per l'ultima volta liberata (se è libera) (v) un booleano che indica se la pagina è sporca (1 indica che è stata modificata dopo il caricamento o dopo l'ultima scrittura).

Pagina del buffer:	0	1	2	3
numero di pin sulla pagina	1	2	0	1
blocco	70	33	35	47
istante load	1	7	3	9
istante unpin			8	
dirty	1	0	0	0

Si supponga ora che vengano eseguite (a partire dall'istante 10, dopo che all'istante 9 è stata caricata la pagina 3) le seguenti operazioni (in cui l'argomento delle pin e unpin è il blocco di interesse, con setXXX si indica un aggiornamento di un qualche valore nel blocco e con flushAll il flush dell'intero buffer):

Riempiendo la tabella seguente, indicare, per ciascuna delle strategie e per ciascuna delle pin, (1) quale pagina del buffer viene utilizzata, (2) se il blocco corrispondente viene letto dal disco e (3) se viene eseguita una scrittura su disco.

	naif			LRU			clock		
Operazione	Pagina	Legge?	Scrive?	Pagina	Legge?	Scrive?	Pagina	Legge?	Scrive?
pin(60)									
pin(70)									
pin(60)									
pin(70)									

10	unpin(70)
11	pin(60)
12	setXXX(60,...)
13	unpin(60)
14	flushAll
15	setXXX(47,...)
16	unpin(47)
17	pin(70)
18	setXXX(70,...)
19	unpin(70)
20	pin(60)
21	unpin(60)
22	pin(70)

Nella figura seguente è schematizzato un piccolissimo buffer con quattro pagine (numerate da 0 a 3), il cui stato viene descritto, per ciascuna pagina, da (i) un intero che indica il numero di pin su di essa (quindi 0 indica che la pagina è libera) (ii) un riferimento al blocco che per ultimo è stato caricato nella pagina; (iii) l'istante in cui è stato effettuato l'ultimo caricamento; (iv) l'istante in cui la pagina è stata per l'ultima volta liberata (se è libera) (v) un booleano che indica se la pagina è sporca (1 indica che è stata modificata dopo il caricamento o dopo l'ultima scrittura).

Pagina del buffer:	0	1	2	3
numero di pin sulla pagina	1	2	0	1
blocco	70	33	35	47
istante load	1	7	3	9
istante unpin			8	
dirty	1	0	0	0

Si supponga ora che vengano eseguite (a partire dall'istante 10, dopo che all'istante 9 è stata caricata la pagina 3) le seguenti operazioni (in cui l'argomento delle pin e unpin è il blocco di interesse, con setXXX si indica un aggiornamento di un qualche valore nel blocco e con flushAll il flush dell'intero buffer):

Riempiendo la tabella seguente, indicare, per ciascuna delle strategie e per ciascuna delle pin, (1) quale pagina del buffer viene utilizzata, (2) se il blocco corrispondente viene letto dal disco e (3) se viene eseguita una scrittura su disco.

	naif			LRU			clock		
Operazione	Pagina	Legge?	Scrive?	Pagina	Legge?	Scrive?	Pagina	Legge?	Scrive?
pin(60)									
pin(70)									
pin(60)									
pin(70)									

10	unpin(70)
11	pin(60)
12	setXXX(60,...)
13	unpin(60)
14	flushAll
15	setXXX(47,...)
16	unpin(47)
17	pin(70)
18	setXXX(70,...)
19	unpin(70)
20	pin(60)
21	unpin(60)
22	pin(70)

NAIF

	0	1	2	3
N. PIN	1	2	0	1
BLOCCO	70	33	35	47
1ST LOAD	1	7	3	9
1ST UNPIN			8	
DIRTY	1	0	0	0

		naif			LRU			clock		
Istante	Operazione	Pagina	Legge?	Scrive?	Pagina	Legge?	Scrive?	Pagina	Legge?	Scrive?
11	pin(60)									
17	pin(70)									
20	pin(60)									
22	pin(70)									

10	unpin(70)
11	pin(60)
12	setXXX(60,...)
13	unpin(60)
14	flushAll
15	setXXX(47,...)
16	unpin(47)
17	pin(70)
18	setXXX(70,...)
19	unpin(70)
20	pin(60)
21	unpin(60)
22	pin(70)

NAIF

	0	1	2	3
N. PIN	1	2	0	0
BLOCCO	70	33	35	47
IST LOAD	22	7	3	9
IST UNPIN			8	16
DIRTY	0	0	0	1

		naif			LRU			clock		
Istante	Operazione	Pagina	Legge?	Scrive?	Pagina	Legge?	Scrive?	Pagina	Legge?	Scrive?
11	pin(60)	0	SI	SI						
17	pin(70)	0	SI	NO						
20	pin(60)	0	SI	SI						
22	pin(70)	0	SI	NO						

10	unpin(70)
11	pin(60)
12	setXXX(60,...)
13	unpin(60)
14	flushAll
15	setXXX(47,...)
16	unpin(47)
17	pin(70)
18	setXXX(70,...)
19	unpin(70)
20	pin(60)
21	unpin(60)
22	pin(70)

NAIF

	0	1	2	3
N. PIN	1	2	0	0
BLOCCO	70	33	35	47
1ST LOAD	22	7	3	9
1ST UNPIN			8	16
DIRTY	0	0	0	1

		naif			LRU			clock		
Istante	Operazione	Pagina	Legge?	Scrive?	Pagina	Legge?	Scrive?	Pagina	Legge?	Scrive?
11	pin(60)	0	SI	SI						
17	pin(70)	0	SI	NO						
20	pin(60)	0	SI	SI						
22	pin(70)	0	SI	NO						

10	unpin(70)
11	pin(60)
12	setXXX(60,...)
13	unpin(60)
14	flushAll
15	setXXX(47,...)
16	unpin(47)
17	pin(70)
18	setXXX(70,...)
19	unpin(70)
20	pin(60)
21	unpin(60)
22	pin(70)

LRU

	0	1	2	3
N. PIN	1	2	0	1
BLOCCO	70	33	35	47
1ST LOAD	1	7	3	9
1ST UNPIN			8	
DIRTY	1	0	0	0

		naif			LRU			clock		
Istante	Operazione	Pagina	Legge?	Scrive?	Pagina	Legge?	Scrive?	Pagina	Legge?	Scrive?
11	pin(60)	0	SI	SI						
17	pin(70)	0	SI	NO						
20	pin(60)	0	SI	SI						
22	pin(70)	0	SI	NO						

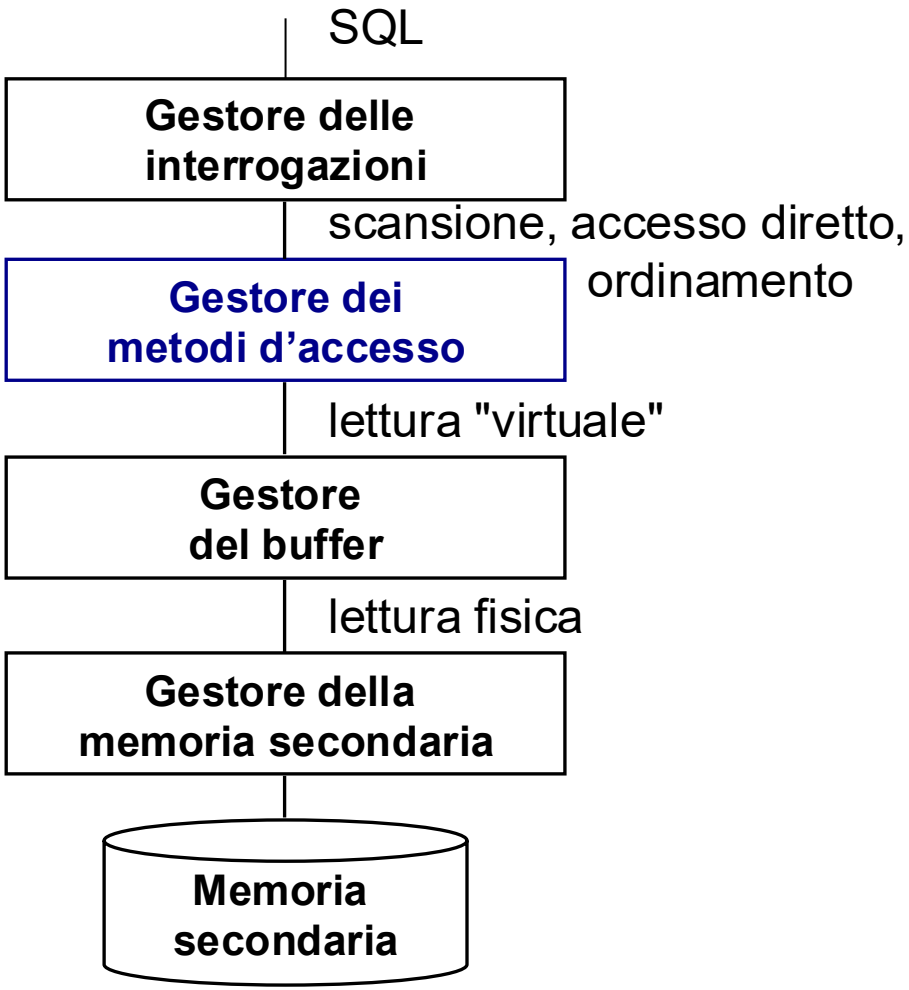
10	unpin(70)
11	pin(60)
12	setXXX(60,...)
13	unpin(60)
14	flushAll
15	setXXX(47,...)
16	unpin(47)
17	pin(70)
18	setXXX(70,...)
19	unpin(70)
20	pin(60)
21	unpin(60)
22	pin(70)

LRU

	0	1	2	3
N. PIN	1	2	0	0
BLOCCO	70	33	60	47
1ST LOAD	1	7	11	9
1ST UNPIN	19		13	16
DIRTY	1	0	0	1

		naif			LRU			clock		
Istante	Operazione	Pagina	Legge?	Scrive?	Pagina	Legge?	Scrive?	Pagina	Legge?	Scrive?
11	pin(60)	0	SI	SI	2	SI	NO			
17	pin(70)	0	SI	NO	0	NO	NO			
20	pin(60)	0	SI	SI	2	NO	NO			
22	pin(70)	0	SI	NO	0	NO	NO			

Gestore degli accessi e delle interrogazioni



In SimpleDB

Remote: riceve le richieste dal client e passa SQL a Planner

Planner: chiama Parser e determina piano e lo passa al Query

Parser: analisi sintattica

Query: riceve il piano e chiama il Record per ogni tabella

Metadata: gestisce gli schemi delle tabelle

Record: gestisce i blocchi per i record delle tabelle

Transaction: gestisce la concorrenza

Buffer: mantiene in memoria le pagine per limitare gli accessi

Log: tiene traccia delle operazioni, per l'affidabilità

File: legge e scrive le pagine su disco

Blocchi e record

- Un file è
  - **logicamente** composto di record (ennuple)
    - la tipica operazione su un file è l'accesso ai record (uno alla volta, anche iterando)
  - **fisicamente** organizzato in blocchi
    - l'operazione eseguita su disco è l'accesso ad un blocco
- I blocchi (componenti "fisici" di un file) e i record (componenti "logici") hanno dimensioni in generale diverse:
  - la dimensione del blocco dipende dal file system
  - la dimensione del record (semplificando un po') dipende dalle esigenze dell'applicazione (ad esempio, gli attributi della relazione), e può anche variare nell'ambito di un file

Record e blocchi, dettagli

- I campi (e quindi i record) possono essere
  - a **lunghezza fissa** (semplifica le operazioni, in particolare le modifiche, ma può causare spreco di spazio)
  - a **lunghezza variabile**
- I blocchi (e i file) possono essere
  - **omogenei** (tutti i record dalla stessa relazione)
  - **eterogenei** (record provenienti da relazioni diverse)
- I record possono essere
  - ognuno interamente in un blocco (il che richiede che i record siano più piccoli dei blocchi): **unspanned**
  - anche divisi fra più blocchi (necessario se sono grandi, utilizzato anche per risparmiare un po' di spazio): **spanned**

Fattore di blocco

- Numero di record in un blocco
- Nel caso, frequente, di blocchi omogenei e record a lunghezza fissa, più piccola della dimensione del blocco:
  - L_R: dimensione di un record
  - L_B: dimensione di un blocco
  - fattore di blocco:

$$\lfloor L_B / L_R \rfloor$$

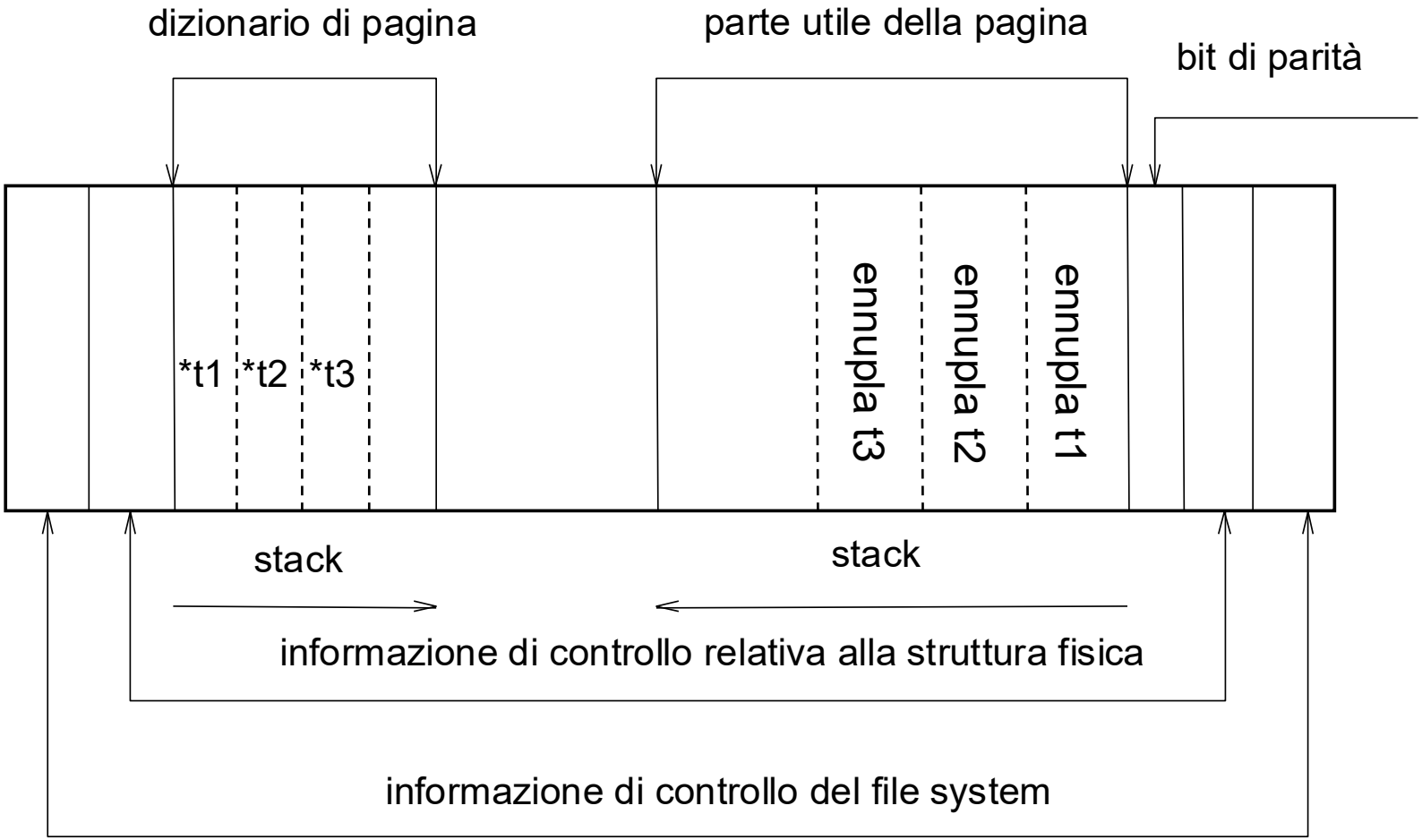
  - approssimato con:

$$L_B / L_R$$

Organizzazione delle ennuple nelle pagine

- Varie alternative, anche legate alle specifiche strutture fisiche
- Vediamo una possibilità (in SimpleDB ne vedremo almeno un'altra), con ennuple "piccole" (caso frequente)
- Nota bene:
  - se la lunghezza dei record è fissa, la struttura può essere semplificata

Organizzazione delle ennuple nelle pagine



Strutture fisiche

- Due aspetti
  - modalità di organizzazione dei record in un file (o delle ennuple di una tabella):
    - strutture primarie
  - ulteriori elementi che permettono un accesso efficiente ai record di un file
    - strutture secondarie

Tipi di strutture fisiche

- Sequenziali
  - primarie
- Calcolate ("Hash")
  - primarie (e talvolta secondarie)
- Ad albero (di solito, indici)
  - secondarie o primarie

Strutture sequenziali

- Esiste un ordinamento fra le ennuple, che può essere rilevante ai fini della gestione
  - **seriale** (o **disordinata**)*:* ordinamento fisico ma non logico
  - **array***:* posizioni individuate attraverso valori (numerici progessivi)
  - **ordinata**: ordinamento fisico coerente con quello di un campo

Una relazione e la sua memorizzazione

- Supponiamo:
  - lunghezza fissa, 2 byte per ID e 6 per Cognome
  - blocco di 20 byte, memorizzazione unspanned

ID	Cognome
CD	Rossi
BC	Bruni
AA	Neri
DA	Verdi
EK	Bisi

Una relazione e la sua memorizzazione

- Supponiamo:
  - lunghezza fissa, 2 byte per ID e 6 per Cognome
  - blocco di 20 byte, memorizzazione unspanned

ID	Cognome
CD	Rossi
BC	Bruni
AA	Neri
DA	Verdi
EK	Bisi

Una relazione e la sua memorizzazione

- Supponiamo:
  - lunghezza fissa, 2 byte per ID e 6 per Cognome
  - blocco di 20 byte, memorizzazione unspanned

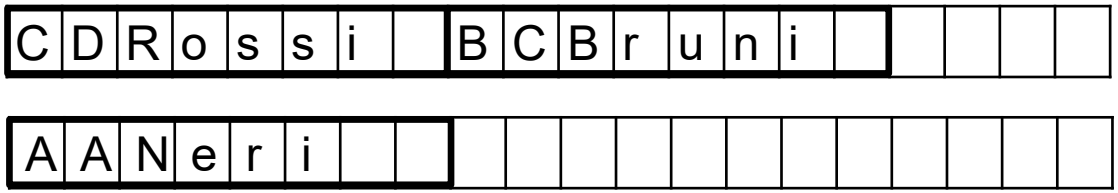
ID	Cognome
CD	Rossi
BC	Bruni
AA	Neri
DA	Verdi
EK	Bisi

C	D	R	o	s	s	i		B	C	B	r	u	n	i					

Una relazione e la sua memorizzazione

- Supponiamo:
  - lunghezza fissa, 2 byte per ID e 6 per Cognome
  - blocco di 20 byte, memorizzazione unspanned

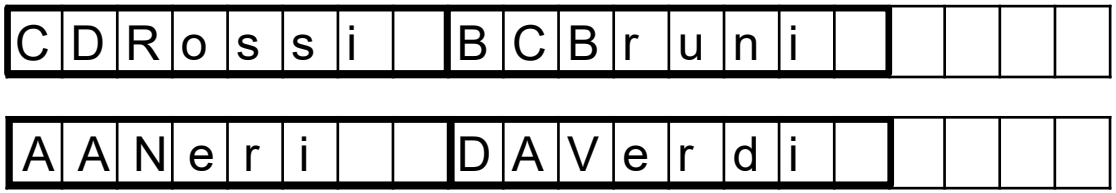
ID	Cognome
CD	Rossi
BC	Bruni
AA	Neri
DA	Verdi
EK	Bisi



Una relazione e la sua memorizzazione

- Supponiamo:
  - lunghezza fissa, 2 byte per ID e 6 per Cognome
  - blocco di 20 byte, memorizzazione unspanned

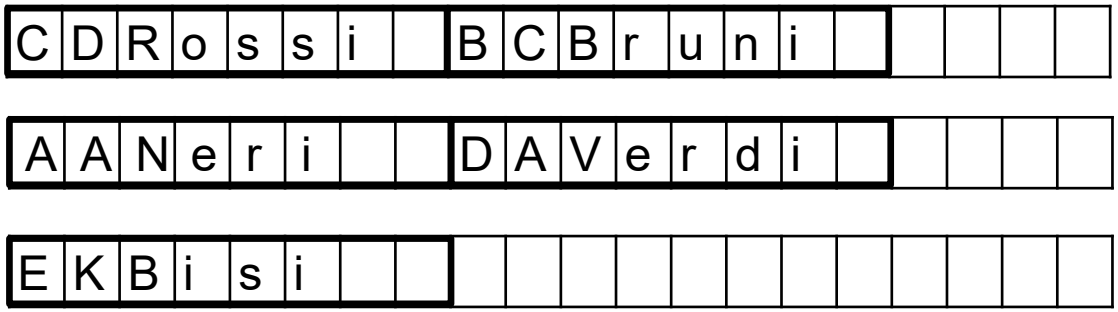
ID	Cognome
CD	Rossi
BC	Bruni
AA	Neri
DA	Verdi
EK	Bisi



Una relazione e la sua memorizzazione

- Supponiamo:
  - lunghezza fissa, 2 byte per ID e 6 per Cognome
  - blocco di 20 byte, memorizzazione unspanned

ID	Cognome
CD	Rossi
BC	Bruni
AA	Neri
DA	Verdi
EK	Bisi



Struttura disordinata

- Chiamata anche:
  - file seriale
  - "entry sequenced"
  - file heap ("mucchio")
- Molto diffusa nelle BD relazionali, associata a indici secondari
- Inserimenti
  - di solito in coda (con riorganizzazioni periodiche)
  - oppure al posto di record cancellati
- Eliminazioni
  - «lasciando i vuoti» (salvo riorganizzazioni periodiche)
- La gestione è molto semplice, ma spesso inefficiente
  - molte varianti

Operazioni su file disordinato

- Scansione
  - beforeFirst() – posiziona all'inizio
  - next() – avanza di un record
  - getInt(fieldname), getString(fieldname) – restituisce il valore di un campo del record corrente
- Aggiornamento
  - setInt(fieldname, value) – modifica il valore di un campo
  - insert() – inserisce un record (ad esempio nel primo posto che trova dopo il record corrente)
  - delete() – elimina il record corrente

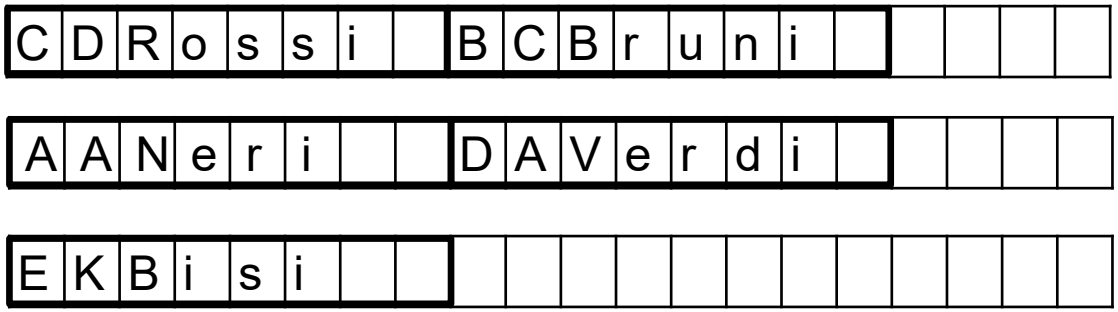
- Nota: esiste quindi un "record corrente"

Le "implementazioni": cenni alla lavagna

- Next:
  - se nel blocco corrente ci sono record non visitati
    - passa al primo di essi
    - altrimenti, passa al primo record del blocco successivo
- Simili le altre, sempre tenendo conto del record corrente

Una relazione e la sua memorizzazione

ID	Cognome
CD	Rossi
BC	Bruni
AA	Neri
DA	Verdi
EK	Bisi



Esercizio

- **esercizio 1 del 30/03/2015**

(Svolto a lezione, vedere soluzioni sul sito dei compiti)

File disordinato, costi

- Sia per ricerca, sia per inserimento:
  - costo di solito lineare (nel numero di blocchi del file)
- Osservazioni (marginali)
  - se il record è presente, il costo sarà mediamente pari alla metà del numero di blocchi, ma possiamo trascurare
  - per l'inserimento, potremmo mantenere un riferimento all'ultimo blocco (però in caso di vincoli da verificare, ad esempio chiave, la scansione ci servirebbe comunque)